Programação Concorrente: Linguagens e Técnicas

Summary of key features

António Ravara

(with Bernardo Toninho and Carla Ferreira)

Departament of Informatics,
NOVA School of Science and Technology

1. Basic Types

Primitive types - basic values

Standard library: `https://doc.rust-lang.org/std/`

- Booleans: `bool = {true,false}`
- Single characters: `char`
- A growable string: `String`
- Numeric:
  - 8 to 128 bits integers (ex: `i32`)
  - 8 to 128 bits unsigned integers (ex: `u32`)
  - 32 or 64 bits floating points (ex: `f32`)
  - architecture dependent (on the word size): (ex: `isize`)
- Reference types: `&T` and `&mut T`
- Pointers (`*const T`, and `*mut T`) available, but unsafe

Primitive types - basic collections

Heterogeneous sequences: `tuple`

Empty (tuple): `unit = {()}`

A fixed-size array: `[T; N]`, for the element type `T`, and the non-negative compile-time constant size `N`.

A growable generic array: `Vec(T)`

Declaration allocates memory: `let mut v = Vec::new()`

Vec is the type and new the creation function

To insert values:

- directly: `v[0] = 1`
- via functions of the type, like `v.push(1)`
- via the `vec!` macro: `let mut v1 = vec![1, 2, 3]`

Structured types – `enums`

User defined sets of values

```
enum Answer = {Yep,Maybe,Nope}
```

Convertion to numerical values

```
fn num_value(ans: Answer) -> f64 {
  match ans {
    Answer::Yep => 1,
    Answer::Maybe => 0.5,
    Answer::Nope => 0,
  }
}
```

Structured types – `Option` types

Sometimes, no value is the right value

```
enum Option<T> {None,Some(T)}
```

- partial functions
- no default value
- no result

```
fn divide(numer: f64, denomin: f64) -> Option<f64> {
  if denomin == 0.0 { None }
  else { Some(numer / denomin) }
}
```

Table of Contents

1. Basic Types
2. Algebraic Data Types
3. Smart Pointers
4. Closures

2. Algebraic Data Types

A generic stack

Recall it in C: prototype, type, and static/dynamic implementation

Defining the type and creating a stack (over `Vec`)

struct Stack<T> = {stack: Vec<T>}

1. let s = Stack { stack: Vec::new() }
2. impl<T> Stack<T> {
   fn new() -> Self {
   Stack { stack: Vec::new() }
   }
   }

`https://www.kirillvasiltsov.com/writing/how-to-write-a-stack-in-rust/`

Implenting the stack (over Vec)

Impl

- A scope to implement functionalities (*methods*) to ADTs defined via structs, enums, or Traits (more later)
- The first parameter of each method is always self – the instance of the struct the method is being called on

```
fn push(&mut self, item:  T) { self.stack.push(item) }
```

The stack may be empty, so the return type is an Option

```
fn pop(&mut self) -> Option<T> { self.stack.pop() }
```

To avoid removing when reading, return a reference

```
fn top(&self) -> Option<&T> { self.stack.last() }
```

Using `Stack`

```
let mut stack:  Stack<isize> = Stack::new();
stack.push(1);
let item = stack.pop(); // Option type
assert_eq!(item.unwrap(), 1); // value 1 inside Some
```

Traits: collections of function/method (like *interfaces*)

Group function/method signatures to define a set of behaviors

```
pub trait Summary { fn summarize(&self) -> String; }
```

We can use the function/method with different types.

```
pub struct News {
   pub headline: String,
   pub author: String,
   pub content: String,
}
```

```
pub struct Tweet {
   pub username: String,
   pub content: String,
   pub reply: String,
}
```

```
impl Summary for News {
   fn summarize(&self) -> String {
      format!("{}, by {}", self.headline, self.author)
   } }
```

https://doc.rust-lang.org/book

Traits: collections of function/method (like *interfaces*)

Group function/method signatures to define a set of behaviors

```
pub trait Summary { fn summarize(&self) -> String; }
```

We can use the function/method with different types.

```
pub struct News {
   pub headline: String,
   pub author: String,
   pub content: String,
}
```

```
pub struct Tweet {
   pub username: String,
   pub content: String,
   pub reply: String,
}
```

```
impl Summary for Tweet {
   fn summarize(&self) -> String {
      format!("{}: {}", self.username, self.content)
   } }
```

https://doc.rust-lang.org/rust-by-example/trait.html

Table of Contents

1. Basic Types
2. Algebraic Data Types
3. Smart Pointers
4. Closures

3. Smart Pointers

Smart Pointers

Pointers as Data Structures

Implemented usually with struct, also implement the traits:

`Deref`: to make instances behave like references

`Drop`: "exception" code to run when instances go out of scope

Key smart pointer types

`Box<T>`: allocates values on the heap

`Rc<T>`: reference counting – enables multiple ownership

`Ref<T>` and `RefMut<T>`: interior mutability – unsafe
enforce borrowing rules at runtime instead of compile-time

Infinite data-structures

Rust insists to know at compile time how much space is needed to store a data-structure

An abstract (integer) stack

```
enum Stack = {
  Emp,
  Push(isize,Stack)
}
```

A concrete stack (1 over 2 over 3)

```
use crate::Stack::{Emp, Push};

fn main() {
  let stk = Push(1, Push(2, Push(3, Emp)));
}
```

`Compile error`: recursive type has infinite size – what?!

Boxes

We need to store a *pointer* to the next value

Box<T> is a pointer

a pointer's size doesn't depend on what it points to

The abstract (integer) stack

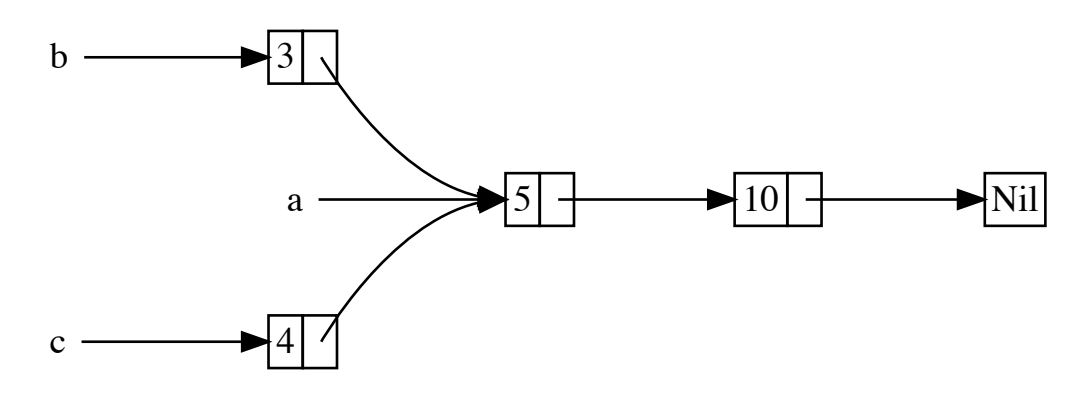
```
enum Stack = {
  Emp,
  Push(isize,Box<Stack>)
}
```

A concrete stack (1 over 2 over 3)

```
fn main() {
  let stk =
Push(1,Box::new(Push(2,Box::new(Push(3,Box::new(Emp))))));
}
```

Reference counting: supports multiple ownership

Consider the following scenario



Can list a be pointed by (shared) by both lists b and c?

No! Once we move a to b we cannot move it again to c.

Rc instead of Box to build graph-like acyclic structures

```
enum List { Nil, Cons(isize, Rc<List>) }

use crate::List::{Nil,Cons}; use std::rc::Rc;

fn main() {
   let a = Rc::new(Cons(5, Rc::new(Cons(10, Rc::new(Nil)))));
   let b = Cons(3, Rc::clone(&a)); // a.clone() is also ok
   let c = Cons(4, Rc::clone(&a));
}
```

- `b` contains a clone (does not own) of the Rc<List> `a` holds

- `c` also contains a clone of the Rc<List> `a` holds

The list is shared by `a`, `b`, and `c`

- Each call Rc::clone increases the reference count to the data within the Rc<List>
- Data won't be cleaned up unless there are zero references to it.

Table of Contents

1. Basic Types
2. Algebraic Data Types
3. Smart Pointers
4. Closures

4. Closures

Closures – anonymous "functions"

Closures are expressions and return values

read variables, call functions or methods, etc

type declaration optional (except when used as function parameters)

Basics, by example

- |n| n+1
  n is the parameter; n+1 the body (evaluating it gives a value)
- |n m| (n+m)*(n+m)
- || 42

Using closures

- let FortyTwo = || 42;
  println!("Print closure FortyTwo: {}", FortyTwo())
- let succ = |n| n+1;
  println!("Print successor of 42: {}", succ(FortyTwo()))

Closures are not functions

Closures capture their environment, by (mut) ref or value

keep values in the scope where the closure is defined to use later

```
let mut count = 0;
let mut inc_and_print = || {
   count += 1;
   println!("count = {}",count);
}
```

Closure may need to take ownership of captured variables

```
let haystack = vec![1, 2, 3];
let contains = move |needle| haystack.contains(needle);
println!("{}", contains(&1));
println!("{}", contains(&4));
```

Disclaimer: there is a lot more to know!

We presented a first introduction to powerful features

Check more info on:

Lifetimes (prevent dangling references)
`https://doc.rust-lang.org/book/ch10-03-lifetime-syntax.html`

Closures (and their (functional) combination with iterators)
`https://doc.rust-lang.org/book/ch13-01-closures.html`

Generic Data Types
`https://doc.rust-lang.org/book/ch10-01-syntax.html`

Traits
`https://doc.rust-lang.org/book/ch10-02-traits.html`